

Lidar Tracking

Lidar Tracking

1. Course Content
2. Start the Agent
2. Run the Sample Program
3. Source Code Analysis

1. Course Content

- Run the sample program to detect the closest object to the vehicle in real-time and control the chassis to track it.

2. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

2. Run the Sample Program

- Start the Lidar tracking node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Enter the command,

```
ros2 launch m3_bringup lidar_demo.launch.py demo:=laser_Tracker
```

After the program starts, the Lidar will scan for the closest object within its detection range. When the object is moved slowly, the program will control the chassis to move, maintaining a distance of 0.5 meters from it and keeping the front of the car aligned with the object.

3. Source Code Analysis

- Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/lidar/laser_Tracker.py
```

- Import necessary libraries:

```
#ros lib
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
#common lib
import math
import numpy as np
import time
from time import sleep
import os
```

Program initialization and creation of publishers and subscribers:

```
def __init__(self, name):
    super().__init__(name)
    # Create a subscriber to the fused Lidar topic messages
    self.sub_laser =
self.create_subscription(LaserScan, "/scan", self.registerScan, 1)
    # Create a subscriber to the joystick remote control topic messages
    self.sub_JoyState = self.create_subscription(Bool, '/JoyState',
self.JoyStateCallback, 1)
    #create a pub
    # Create a publisher for the velocity topic messages
    self.pub_vel = self.create_publisher(Twist, '/cmd_vel', 1)
    #declareparam
    self.declare_parameter("linear", 0.5)
```

```

self.linear =
self.get_parameter('linear').get_parameter_value().double_value
self.declare_parameter("angular",1.0)
self.angular =
self.get_parameter('angular').get_parameter_value().double_value
# Lidar detection angle
self.declare_parameter("LaserAngle",45.0)
self.LaserAngle =
self.get_parameter('LaserAngle').get_parameter_value().double_value
# Obstacle detection distance, a value less than this indicates an obstacle
self.declare_parameter("ResponseDist",0.55)
self.ResponseDist =
self.get_parameter('ResponseDist').get_parameter_value().double_value
# Joystick control flag, if True, the car is controlled by the joystick.
Press R2 to take or enable joystick control.
self.Joy_active = False
# Create a linear velocity PID control object
self.lin_pid = SinglePID(1.0, 0.0, 1.0)
# Create an angular velocity PID control object
self.ang_pid = SinglePID(2.0, 0.0, 2.0)

```

`registerscan` Lidar topic callback function:

```

def registerscan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    ranges = np.array(scan_data.ranges)
    minDistList = []
    minDistIDList = []
    for i in range(len(ranges)):
        # Convert radians from Lidar data to degrees
        angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
        # If the current angle is within the detection range and the distance is
not 0
        if abs(angle) < self.LaserAngle and ranges[i] !=0.0 :
            # Add the distance at this angle to the minimum distance list
            minDistList.append(ranges[i])
            # Add this angle to the minimum ID list
            minDistIDList.append(angle)
        # If the minimum distance list is not empty, it means the closest object has
been detected
        if len(minDistList) != 0:
            # Find the minimum value in the distance list
            minDist = min(minDistList)
            # Find the angle corresponding to the shortest distance
            minDistID = minDistIDList[minDistList.index(minDist)]
        else:
            #self.pub_vel.publish(Twist())
            print("-----")
            return
        # If self.Joy_active is True, publish a stop velocity and exit the callback
        if self.Joy_active :
            self.pub_vel.publish(Twist())
            return
    velocity = Twist()
    print("minDist: ",minDist)
    print("minDistID: ",minDistID)

```

```

# If the difference between the minimum distance and the set value is less
than 0.1m, update the current minimum distance to the set value
if abs(minDist - self.ResponseDist) < 0.1: minDist = self.ResponseDist
# Calculate linear velocity
velocity.linear.x = -self.lin_pid.pid_compute(self.ResponseDist, minDist)
# Calculate angular velocity
ang_pid_compute = self.ang_pid.pid_compute(abs(minDistID) / 72, 0)
# If the minimum angle is greater than 0, it means the object is on the left,
so the car turns right
if 0 < minDistID :
    velocity.angular.z = ang_pid_compute
# If the minimum angle is less than 0, it means the object is on the right,
so the car turns left
elif minDistID < 0:
    velocity.angular.z = -ang_pid_compute
if abs(ang_pid_compute) < 0.5: velocity.angular.z = 0.0
velocity.angular.z = velocity.angular.z * 0.6
print("angular.z: ", velocity.angular.z)
# Publish the velocity control topic
self.pub_vel.publish(velocity)

```